

성장트리

학생 정보 관리 시스템

프로젝트 보고서



과목명	자료구조
담당교수	정명희 교수
학과	소프트웨어학과
학번	2022E7415 / 2025E7330
이름	원종찬 / 주예서
제출 일자	2026.06.10

목 차

I. 서론

1. 개발 계획서	3
2. 문제 정의 및 분석	3

II. 본론

1. 알고리즘 설계	4
2. 코드 설명	5
3. 코드 작동 화면 캡처 및 설명	9

III. 결론

1. 소감 및 새롭게 학습한 내용 정리	11
-----------------------------	----

I. 서론

1. 개발 계획서

본 프로젝트의 개발 목표는 C 언어를 기반으로 이진 탐색 트리(BST) 및 AVL 트리, 해시 맵, 계층형 트리를 결합한 학생 정보 관리 시스템을 구현하는 것이다. 단순한 데이터 저장을 넘어 성적 분석 및 학생 분류까지 지원하는 통합 관리 시스템을 완성하고, 이 과정을 통해 다양한 자료구조의 원리와 실제 적용 방법을 체득하는 것을 목표로 한다.

전체 시스템은 크게 세 개의 레이어로 구성된다. 최상단의 사용자 인터페이스 레이어는 메뉴 출력과 입력 처리를 담당하고, 중간층의 핵심 기능 레이어는 등록, 조회, 성적 관리, 학생 분류 등의 주요 기능을 수행한다. 최하단의 자료구조 레이어는 계층형 트리나 해시 맵으로 구성되어 데이터를 실제로 저장하고 관리한다. 학사 조직은 학교를 최상위 노드로 하여 학과, 학년, 반이 순차적으로 연결되는 계층형 트리 구조로 표현되며, 각 반 노드에는 해당 반 학생들의 연결 리스트가 연결된다. 해시 맵은 학번을 키로 사용하며, 해시 함수로 계산된 인덱스에 학생 노드의 주소를 저장하고 충돌은 Chaining 방식으로 처리한다.

1주차에는 프로젝트 기획 및 구조 설계를 진행한다. 요구사항 분석 및 기능 목록 설정을 공동으로 수행하고, 자료구조 설계 및 구조체 정의와 GitHub 레포지토리 생성 및 브랜치 전략 수립을 각각 분담하여 진행한다. 주차 말에는 전체 코드 골격을 공동으로 작성한다. 2주차에는 관리 시스템 기본 기능을 구현한다. 계층형 트리 생성 및 노드 삽입, 삭제, 학생 등록 및 해시 맵 연동 구현과 해시 맵 구현 및 학번 기반 조회 기능을 각각 분담하여 진행하며, 주차 말에는 공동으로 단위 테스트 및 버그 수정을 진행한다. 3주차에는 확장 기능 및 트리 기능을 구현한다. 성적 입력, 수정, 평균 점수 계산 기능 구현과 AVL 트리 회전 알고리즘 및 우수, 위험 학생 분류 기능 구현을 각각 분담하여 진행하며, 주차 말에는 전체 기능 통합 테스트를 수행한다. 4주차에는 최종 점검 및 발표 준비를 진행한다. 메모리 누수 점검 및 예외 처리 보완과 코드 리팩토링 및 주석 정리를 각각 분담하여 진행하며, 이후 최종 보고서 작성과 발표 자료 준비 및 발표 연습을 공동으로 진행한다.

2. 문제 정의 및 문제 분석

현대 교육 환경에서 학생 수는 점점 증가하고 있으며, 이에 따라 학생 정보를 효율적으로 저장하고 관리하는 시스템의 필요성이 높아지고 있다. 기존의 단순 배열 또는 선형 리스트 기반의 학생 정보 관리 방식은 소규모 데이터에서는 문제없이 동작하지만, 데이터의 양이 증가할수록 탐색, 삽입, 삭제 연산의 시간 복잡도가 $O(n)$ 에 달하여 성능이 급격히 저하되는 한계를 보인다. 또한 학교 내 조직은 학교, 학과, 학년, 반으로 이어지는 명확한 계층 구조를 가지고 있음에도 불구하고, 기존 방식은 이 계층성을 반영하지 못하고 데이터를 평면적으로 저장하는 경우가 많다. 이로 인해 특정 학과나 반 단위로 학생 정보를 일괄 조회하거나 분석하는 작업이 어렵고 비효율적이다. 이러한 문제를 해결하기 위해 본 프로젝트에서는 계층형 트리 구조와 해시 맵을 결합한 학생 정보 관리 시스템을 설계하고 구현한다. 트리 구조를 통해 학교 조직의 계층성을 자연스럽게 표현하고, 해시 맵을 통해 학번 기반 $O(1)$ 탐색을 실현함으로써 대규모 데이터 환경에서도 빠르고 안정적인 정보 관리를 가능하게 한다.

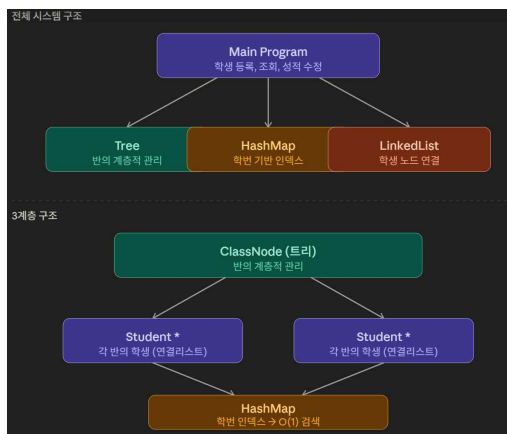
본 프로젝트가 해결하고자 하는 핵심 문제는 다음 세 가지로 정의한다. 첫 번째는 선형 구조의 탐색 비효율성이다. 기존 배열이나 연결 리스트 기반 시스템에서는 특정 학생을 찾기 위해 처음부터 끝까지 순차 탐색을 수행해야 한다. 학생 수가 n 명일 때 최악의 경우 $O(n)$ 의 시

간이 소요되며, 수천 명 이상의 학생 데이터를 다루는 실제 학교 환경에서는 응답 속도가 크게 저하된다. 두 번째는 계층 구조 미반영으로 인한 관리 불편이다. 학교 조직은 본질적으로 계층적이며, 특정 학과 또는 특정 반의 학생 전체를 조회하거나 성적을 분석할 때 계층 구조가 반영되지 않은 시스템에서는 전체 데이터를 순회하며 조건을 일일이 확인해야 하는 비효율이 발생한다. 세 번째는 성적 데이터의 분석 기능 부재이다. 단순 정보 저장에 그치는 기존 방식은 학생별 평균 성적 계산, 우수 학생 및 학업 위험 학생 분류 등의 분석 기능을 제공하지 못한다. 이로 인해 교육 현장에서 학생 지도에 필요한 데이터 기반 의사결정이 어렵다.

위의 문제 정의를 바탕으로 시스템의 요구사항을 기능적 요구사항과 비기능적 요구사항으로 구분하여 분석한다. 기능적 요구사항으로는 학교, 학과, 학년, 반 노드를 생성하고 삭제할 수 있는 학사 조직 계층 구조 관리 기능, 학생 정보를 특정 반에 등록하고 학번을 키로 해시 맵에 자동 등록하는 학생 등록 및 해시 인덱싱 기능, 학번 입력만으로 $O(1)$ 시간 내에 학생 정보를 반환하는 즉시 조회 기능, 학생별 과목 점수를 입력하고 수정할 수 있는 성적 관리 기능, 개인 및 반 전체의 평균 점수를 계산하는 기능, 그리고 평균 점수 기준으로 우수 학생과 학업 위험 학생을 자동 분류하는 기능이 포함된다. 비기능적 요구사항으로는 학번 기반 탐색 $O(1)$, 트리 탐색 $O(\log n)$ 을 목표로 하는 탐색 성능, 학과 및 반의 수가 증가하더라도 노드 추가만으로 확장 가능한 확장성, 동적 메모리 할당을 활용한 메모리 효율, 그리고 각 자료구조와 기능을 모듈화하여 독립적으로 수정 가능하도록 하는 유지보수성이 요구된다.

II. 본론

1. 알고리즘 설계



전체 시스템 구조는 Main Program을 최상단에 두고 그 아래 Tree, HashMap, LinkedList 세 가지 핵심 자료구조가 연결되는 형태로 구성된다. Main Program은 학생 등록, 조회, 성적 수정 등 사용자의 모든 요청을 받아 처리하는 중심 역할을 하며, 각 자료구조에 필요한 작업을 위임하는 방식으로 동작한다. 이러한 구조는 각 모듈이 독립적으로 기능을 수행하면서도 유기적으로 연결되어 있어 유지보수성과 확장성을 동시에 확보할 수 있다는 장점이 있다.

계층 구조의 최상단은 ClassNode로, 트리 구조를 기반으로 학교, 학과, 학년, 반으로 이어지는 학사 조직의 계층적 관계를 표현한다. ClassNode는 각 반을 하나의 노드로 관리하며, 반 단위로 학생 정보를 묶어 체계적으로 저장할 수 있도록 설계된다. 이를 통해 특정 학과나 반 전체의 학생 정보를 일괄적으로 조회하거나 분석하는 작업이 직관적이고 효율적으로 이루어질 수 있다.

ClassNode 하위에는 각 반에 소속된 학생들을 저장하는 Student 연결 리스트가 연결된다. 학생 노드들은 포인터로 순차적으로 이어지는 구조를 통해 동적으로 학생을 추가하거나 삭제할 수 있으며, 고정된 크기를 미리 할당할 필요 없이 학생 수의 변화에 유연하게 대응할 수 있다. 연결 리스트 방식은 메모리를 효율적으로 활용할 수 있어 실제 학교 환경처럼 학생 수가 빈번하게 변동되는 상황에서도 안정적으로 동작한다.

모든 Student 노드는 최하단의 HashMap과 연결된다. HashMap은 학번을 키로 사용하여 각 학생 노드의 주소를 저장하고 있으며, 트리 전체를 순회하지 않고도 학번 입력만으로 $O(1)$ 시간 내에 해당 학생 정보에 즉시 접근할 수 있도록 한다. 이는 학생 수가 증가하더라도 탐색 속도가 저하되지 않는다는 점에서 시스템의 탐색 성능을 크게 향상시키는 핵심 설계이며, 계층형 트리 구조와 결합하여 구조적 관리와 빠른 검색이라는 두 가지 목표를 동시에 달성한다.

2. 코드 설명

```

1  #ifndef STUDENT_H
2  #define STUDENT_H
3
4  #define NAME_LEN 50
5
6  typedef struct Student {
7      int id;
8      char name[NAME_LEN];
9
10     int korean;
11     int english;
12     int math;
13
14     float average;
15
16     struct Student* next;
17 } Student;
18
19 Student* createStudent(
20     int id,
21     char* name,
22     int kor,
23     int eng,
24     int math);
25
26 void printStudent(
27     Student* student);
28
29 #endif

```

student.h 파일은 학생 정보를 나타내는 Student 구조체와 관련 함수의 선언이 포함되어 있다. 헤더 파일 보호를 위한 `#ifndef STUDENT_H`, `#define STUDENT_H`, `#endif`가 포함되며, `#define NAME_LEN 50`으로 이름의 최대 길이를 정의한다. Student 구조체는 학생의 학번(id), 이름(name), 국어/영어/수학 성적(korean/english/math), 평균(average), 그리고 연결리스트의 다음 노드를 가리키는 포인터(next)로 구성된다. 이 구조체는 각 학생의 모든 정보를 포함하며, next 포인터를 통해 같은 반의 학생들을 연결리스트로 연결한다. 파일의 끝에는 `createStudent()` 함수로 새로운 Student 객체를 생성하고, `printStudent()` 함수로 학생 정보를 출력하는 두 가지 함수가 선언된다.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "student.h"
6
7  Student* createStudent(
8      int id,
9      char* name,
10     int kor,
11     int eng,
12     int math)
13 {
14     Student* student =
15         (Student*)malloc(sizeof(Student));
16
17     if (student == NULL)
18     {
19         printf("메모리 할당 실패\n");
20         return NULL;
21     }
22
23     student->id = id;
24
25     strcpy(student->name, name);
26
27     student->korean = kor;
28     student->english = eng;
29     student->math = math;

```

student.c 파일은 student.h에서 선언한 createStudent() 함수와 printStudent() 함수를 구현한다. createStudent() 함수는 학번, 이름, 국어/영어/수학 성적을 매개변수로 받아 malloc()으로 메모리를 동적으로 할당하고, 할당 실패 시 NULL을 반환한다. 할당에 성공하면 각 필드에 값을 저장하고, 특히 평균값을 (kor + eng + math) / 3.0f로 계산하여 저장합니다(3.0으로 표기하여 부동소수점 연산 보장). printStudent() 함수는 Student 포인터를 받아 NULL 검사 후 학번, 이름, 각 과목 성적, 그리고 %.2f 포맷으로 소수점 둘째 자리까지의 평균을 화면에 출력한다. 이 두 함수는 학생 객체의 생성과 정보 표시를 담당하는 기본적인 작업을 수행한다.

```

1  #ifndef TREE_H
2  #define TREE_H
3
4  #include "student.h"
5
6  typedef struct ClassNode {
7      char className[50];
8
9      Student* students;
10
11     struct ClassNode* left;
12     struct ClassNode* right;
13 } ClassNode;
14
15 ClassNode* createClass(char* name);
16
17 void insertClass(
18     ClassNode** root,
19     char* name);
20
21 ClassNode* searchClass(
22     ClassNode* root,
23     char* name);
24
25 void inorderClass(
26     ClassNode* root);
27
28 void addStudent(
29     ClassNode* classNode,
30     Student* student);
31
32 float calculateClassAverage(
33     ClassNode* classNode);
34

```

tree.h 파일은 반(Class) 정보를 관리하는 이진 탐색 트리(BST)와 관련된 구조체 및 함수 선언을 포함한다. 헤더 가드와 #include "student.h"를 통해 Student 구조체를 참조한다. ClassNode 구조체는 반의 이름(className), 해당 반의 학생 연결리스트 헤드(students), 그리고 이진 탐색 트리의 왼쪽/오른쪽 자식 포인터(left/right)로 구성되며, 반 이름의 알파벳 순서에 따라 정렬되어 저장됩니다. 파일의 하단에는 createClass(), insertClass(), searchClass(), inorderClass(), addStudent(), calculateClassAverage(), printExcellentStudents(), printRiskStudents() 총 8개의 함수가 선언되어 있으며, 이들은 반의 생성, 삽입, 검색, 순회, 학생 추가, 평균 계산, 우수/위험 학생 조회 등의 역할을 수행한다.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "tree.h"
6
7  ClassNode* createClass(char* name)
8  {
9      ClassNode* node =
10         (ClassNode*)malloc(sizeof(ClassNode));
11
12         if (node == NULL)
13         {
14             printf("메모리 할당 실패\n");
15             return NULL;
16         }
17
18         strcpy(node->className, name);
19
20         node->students = NULL;
21
22         node->left = NULL;
23         node->right = NULL;
24
25         return node;
26     }
27
28     void insertClass(
29         ClassNode* root,
30         char* name)
31     {
32         if (*root == NULL)
33         {
34             *root = createClass(name);

```

tree.c 파일은 tree.h에서 선언한 모든 함수들을 구현한다. createClass() 함수는 malloc으로 새로운 ClassNode를 생성하고 반 이름을 저장한 후 students와 자식 포인터들을 NULL로 초기화한다. insertClass() 함수는 더블 포인터를 사용하는 재귀 함수로, strcmp로 반 이름을 비교하여 왼쪽 또는 오른쪽 서브트리에 삽입한다. searchClass() 함수는 strcmp 비교를 통해 이진 탐색으로 특정 반을 $O(\log n)$ 시간에 검색한다. inorderClass() 함수는 중순 순회로 모든 반을 알파벳 순서로 출력한다. addStudent() 함수는 새로운 학생을 반의 학생 연결 리스트의 맨 앞에 $O(1)$ 시간에 삽입한다. calculateClassAverage() 함수는 반의 모든 학생을 순회하면서 각 학생의 미리 계산된 평균을 더해 반의 평균을 구한다. printExcellentStudents() 함수는 평균이 80 이상인 학생들을 찾아 출력하고, printRiskStudents() 함수는 평균이 60 이하인 학생들을 찾아 출력한다.

```

1  #ifndef HASHMAP_H
2  #define HASHMAP_H
3
4  #include "student.h"
5
6  #define TABLE_SIZE 100
7
8  typedef struct HashNode {
9      int key;
10     Student* student;
11     struct HashNode* next;
12 } HashNode;
13
14 extern HashNode* hashTable[TABLE_SIZE];
15
16 int hashFunction(int key);
17
18 void insertHash(Student* student);
19
20 Student* searchHash(int id);
21
22 #endif

```

hashmap.h 파일은 해시맵 기반의 빠른 학생 검색을 구현하기 위한 구조체와 함수 선언을 포함한다. #define TABLE_SIZE 100으로 해시테이블의 크기를 100개의 버킷으로 정의하며, 이는 충돌을 최소화하면서도 메모리를 효율적으로 사용한다. HashNode 구조체는 학생의 학번(key), 학생 데이터의 포인터(student), 그리고 같은 버킷의 다음 노드를 가리키는 포인터(next)로 구성되며, 이를 통해 해시 충돌 시에도 체이닝으로 여러 학생을 관리할 수 있다. extern HashNode* hashTable[TABLE_SIZE]는 전역 해시테이블을 외부 파일에서 접근할 수 있도록 선언한다. 파일의 하단에는 hashFunction(), insertHash(), searchHash() 세 가지 함수가 선언되어 있으며, 이들은 각각 학번을 인덱스로 변환하고, 학생을 해시테이블에 삽입하며, 학번으로 빠르게 검색하는 역할을 한다.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  #include "hashmap.h"
5
6  HashNode* hashTable[TABLE_SIZE] = { NULL };
7
8  int hashFunction(int key)
9  {
10     return key % TABLE_SIZE;
11 }
12
13 void insertHash(Student* student)
14 {
15     int index = hashFunction(student->id);
16
17     HashNode* newNode =
18         (HashNode*)malloc(sizeof(HashNode));
19
20     if (newNode == NULL)
21     {
22         printf("메모리 할당 실패\n");
23         return;
24     }
25
26     newNode->key = student->id;
27     newNode->student = student;
28
29     newNode->next = hashTable[index];
30     hashTable[index] = newNode;
31 }
32
33 Student* searchHash(int id)
34 {

```

hashmap.c 파일은 hashmap.h에서 선언한 함수들을 구현하며, 가장 먼저 HashNode* hashTable[TABLE_SIZE] = { NULL }으로 전역 해시테이블을 정의하고 모든 버킷을 NULL로 초기화한다. hashFunction() 함수는 매개변수 key를 TABLE_SIZE(100)로 나눈 나머지를 반환하여, 어떤 학번이든 0부터 99 범위의 인덱스로 변환한다. insertHash() 함수는 Student 포인터를 받아 hashFunction으로 인덱스를 계산한 후, 새로운 HashNode를 malloc으로 생성하고 해당 버킷의 맨 앞에 Chaining으로 삽입한다. searchHash() 함수는 학번을 받아 hashFunction으로 인덱스를 계산한 후, 해당 버킷의 연결리스트를 순회하면서 일치하는 학번을 찾아 Student 포인터를 반환하거나 찾지 못하면 NULL을 반환한다. 이 파일은 해시맵을 통해 학번으로 학생을 극도로 빠르게 검색할 수 있게 한다.

```

1  #ifndef MENU_H
2  #define MENU_H
3
4  void showMenu();
5
6  #endif

```

menu.h 파일은 사용자 인터페이스 관련 함수 선언을 포함하는 간단한 헤더 파일이다. 헤더 가드를 포함하며, void showMenu(void) 함수 하나를 선언한다. 이 함수는 프로그램의 메뉴를 화면에 출력하는 역할을 담당한다.

```

1  #include <stdio.h>
2
3  #include "menu.h"
4
5  void showMenu()
6  {
7     printf("\n");
8     printf("SeongJangTree\n");
9     printf("1. 학생 등록\n");
10    printf("2. 학생 조회\n");
11    printf("3. 반 평균 계산\n");
12    printf("4. 우수 학생 조회\n");
13    printf("5. 위험 학생 조회\n");
14    printf("0. 종료\n");
15    printf("선택 : ");
16 }

```

menu.c 파일은 menu.h에서 선언한 showMenu() 함수를 구현한다. 함수는 프로그램의 제목 "SeongJangTree"를 출력한 후, 1부터 6까지의 기능(학생 등록, 학생 조회, 성적 수정, 평균 계산, 우수 학생 조회, 위험 학생 조회)과 0(종료)을 메뉴 형식으로 화면에 출력한다. 마지막으로 "선택 : "을 출력하여 사용자가 번호를 입력할 수 있도록 유도한다. 이 함수는 메인 함수

수의 무한 루프에서 매번 호출되어 사용자에게 선택 옵션을 반복적으로 제시한다.

```

1  #include <stdio.h>
2
3  #include "student.h"
4  #include "tree.h"
5  #include "hashmap.h"
6  #include "menu.h"
7
8  int main()
9  {
10     ClassNode* root = NULL;
11
12     insertClass(&root, "1-A");
13
14     int choice;
15
16     while (1)
17     {
18         showMenu();
19
20         scanf("%d", &choice);
21
22         switch (choice)
23         {
24             case 1:
25             {
26                 int id;
27                 char name[50];
28                 int kor, eng, math;
29
30                 printf("학번 : ");
31                 scanf("%d", &id);
32
33                 printf("이름 : ");
34                 scanf("%s", name);

```

main.c 파일은 프로그램 전체의 흐름을 제어하는 메인 함수를 포함한다. 함수는 ClassNode* root = NULL로 트리의 루트를 초기화하고, insertClass(&root, "1-A")로 기본 반을 생성한다. while (1)의 무한 루프 내에서 showMenu()로 메뉴를 출력하고 scanf()로 사용자 선택을 입력받은 후, switch 문으로 선택에 따른 작업을 수행한다. case 1에서는 사용자로부터 학번, 이름, 성적을 입력받아 createStudent()로 학생을 생성하고, addStudent()로 트리의 반에 추가하며, insertHash()로 해시테이블에도 추가한다. case 2에서는 searchHash()로 O(1) 시간에 학생을 검색하여 printStudent()로 출력한다. case 3은 미구현, case 4는 calculateClassAverage()로 반 평균을 계산하여 출력, case 5는 printExcellentStudents()로 우수 학생을 출력, case 6은 printRiskStudents()로 위험 학생을 출력, case 0은 "프로그램 종료"를 출력하고 return으로 종료, default는 잘못된 입력을 처리한다. 이 메인 함수는 세 가지 자료구조(트리, 해시맵, 연결리스트)를 통합하여 사용자 친화적인 학생 관리 시스템을 구현한다.

3. 코드 작동 화면 캡처 및 설명

```

SeongJangTree
1. 학생 등록
2. 학생 조회
3. 학생 성적 수정
4. 반 평균 계산
5. 우수 학생 조회
0. 종료
선택 : 1
학번 : 2025
이름 : 주예서
국어 : 90
영어 : 85
수학 : 100
학생 등록 완료

SeongJangTree
1. 학생 등록
2. 학생 조회
3. 학생 성적 수정
4. 반 평균 계산
5. 우수 학생 조회
0. 종료
선택 : 2
검색할 학번 : 2025

학생 정보
학번 : 2025
이름 : 주예서
국어 : 90
영어 : 85
수학 : 100
평균 : 91.67

```

사용자가 1을 입력하면 학생 등록 기능이 실행된다. 프로그램은 학번, 이름, 국어, 영어, 수학 점수를 차례대로 요청한다. 사용자가 각 정보를 입력하면, 프로그램은 세 과목 점수로부터 평균을 자동으로 계산한다. 생성된 학생은 addStudent() 함수를 통해 트리의 "1-A" 반에 연결리스트로 추가되고, insertHash() 함수를 통해 해시테이블의 적절한 버킷에 추가된다. "학생 등록 완료" 메시지가 출력되면 메뉴로 돌아가며, 사용자는 같은 방식으로 여러 명의 학생을 연속적으로 등록할 수 있다. 각 학생이 등록될 때마다 평균이 자동으로 계산되고, 트리의 반에는 맨 앞에 추가되며, 해시테이블에도 추가되므로, 학생 정보가 두 가지 자료구조에 동시에 저장되어 유연한 검색이 가능해진다.

사용자가 2를 입력하면 학생 조회 기능이 실행된다. 프로그램은 검색할 학번을 묻고, 사용자가 학번을 입력하면 프로그램은 searchHash() 함수를 호출한다. 이 함수는 해시 함수로 버킷 인덱스를 구한 후, 해당 버킷의 연결리스트를 순회하면서 입력된 학번과 일치하는 노드를 찾는다. 찾으면 해당 Student 포인터를 반환하고, printStudent() 함수로 학생의 모든 정보를 화면에 출력한다. 이 검색은 평균적으로 $O(1)$ 시간에 완료되므로, 대량의 학생 정보에서도 순식간에 검색할 수 있다. 만약 존재하지 않는 학번을 검색하면, searchHash() 함수는 NULL을 반환하고, printStudent() 함수의 안전성 검사가 친화적인 메시지를 출력한다. 따라서 프로그램이 강제로 종료되지 않고 안전하게 오류를 처리한다.

사용자가 3을 입력하면 성적 수정 기능이 실행된다. 현재 버전에서는 성적 수정 기능이 미구현 상태이므로, 안내 메시지만 출력되고 메뉴로 돌아간다. 이 기능은 향후 개발 계획에 포함되어 있다. 성적 수정 기능이 구현될 때는 학생의 성적을 변경할 때마다 평균값을 재계산하고, 이를 트리의 해당 반과 해시맵의 해당 노드 모두에서 반영해야 하므로, 복잡한 로직이 필요하다.

사용자가 4를 입력하면 반의 전체 평균 점수를 계산한다. 프로그램은 calculateClassAverage(root) 함수를 호출하여 트리의 루트 노드에 해당하는 반의 모든 학생을 순회한다. 이 함수는 반의 각 학생의 미리 계산된 평균값들을 모두 더한 후, 학생 수로 나누어 반의 평균을 계산한다. 계산 결과는 소수점 둘째 자리까지만 표시된다. 반 평균은 반의 전체 학습 수준을 파악할 수 있는 중요한 지표이며, 교사가 반의 성취도를 신속하게 파악하는 데 도움이 된다.

사용자가 5를 입력하면 평균이 80 이상인 우수 학생들을 조회한다. 프로그램은 printExcellentStudents(root) 함수를 호출하여 반의 모든 학생을 순회하면서 각 학생의 평균을 확인한다. 평균이 80 이상인 모든 학생을 학번, 이름, 평균과 함께 출력한다. 이는 연결리스트를 선형으로 순회하면서 조건을 만족하는 학생만 필터링하는 방식이다. 이 기능을 통해 교사는 반의 우수 학생들을 빠르게 파악할 수 있으며, 우수 학생 지원, 장학금 대상자 선정, 수업 심화 프로그램 대상자 결정 등에 활용할 수 있다.

사용자가 6을 입력하면 평균이 60 이하인 위험 학생들을 조회한다. 프로그램은 printRiskStudents(root) 함수를 호출하여 반의 모든 학생을 순회하면서 각 학생의 평균을 확인한다. 평균이 60 이하인 모든 학생을 학번, 이름, 평균과 함께 출력한다. 이는 우수 학생 조회와 동일한 방식이지만 조건이 반대인 것이다. 이 기능을 통해 교사는 학습 지원이 필요한 학생들을 빠르게 파악할 수 있으며, 추가 교육, 학습 상담, 보충 수업 대상자를 결정할 수 있다.

사용자가 0을 입력하면 프로그램이 종료된다. 메인 함수의 switch 문의 case 0이 실행되면 종료 메시지가 출력되고, return 0이 실행되어 main 함수가 종료된다. 이는 OS에 프로그램이 정상적으로 완료되었음을 신호한다. 프로그램이 종료되면 터미널로 돌아가며, 사용자는 다시 다

른 명령어를 입력할 수 있다.

Ⅲ. 결론

1. 소감 및 새롭게 학습한 내용 정리

① 원종찬

이번 학사 관리 시스템 개발 과제는 단순한 문법 습득을 넘어, 다양한 고급 자료구조를 하나의 유기적인 프로그램으로 결합해 보는 도전적인 경험이었다. 처음에는 교수님께서 제시해 주신 명세서의 대규모 계층 구조와 복잡한 담당 함수들을 보고 설계 방향을 잡는 데 많은 고민이 필요했다. 특히 초기 설계 단계에서 요구사항을 완벽히 파악하지 못해 데이터 포맷을 수정해야 하거나, 메뉴 연동 과정에서 의도치 않은 예외 상황들을 마주하기도 했다.

가장 기억에 남는 것은 사용자 편의성을 높이기 위한 예외 처리 과정이었다. 이 과정에서 0을 입력했을 때 사용자가 유연하게 이전 단계(학번 입력)로 돌아갈 수 있도록 제어 흐름을 다듬었다. 코드가 완벽히 빌드되는 것을 넘어, '사용자가 쓰기 편하고 런타임 에러에 안전한 프로그램'을 만드는 것이 왜 중요한지 깊이 깨달았다. 단순한 요구사항 구현에 그치지 않고 설계자 관점에서 예외 처리를 하나씩 해결해 나가는 과정에서 큰 보람을 느꼈다.

본 프로젝트를 통해 대규모 데이터를 체계적으로 관리하기 위한 복합 자료구조의 설계와 연동 방법을 깊이 있게 학습했다. 우선 insertClass 함수를 구현하며 학교, 학과, 반으로 이어지는 복합적인 학사 조직을 일반 이진 탐색 트리 구조로 설계해 보았다. 이를 통해 방대한 계층형 데이터를 코드로 체계화하고 동적으로 조직의 분기를 트는 메커니즘을 익힐 수 있었다.

또한, 트리가 커질수록 탐색 속도가 저하될 수 있다는 단점을 보완하기 위해 searchHash 함수를 도입했다. 학번을 고유한 Key로 활용하는 해시 함수와 충돌을 방지하는 체이닝 기법을 결합함으로써, 전체 데이터를 일일이 순회하지 않고도 $O(1)$ 의 시간 복잡도로 원하는 학생 정보를 즉시 찾아내는 해싱 최적화 원리를 확실히 체득했다.

마지막으로, 학생들의 학점 데이터를 실시간으로 정렬하고 효율적으로 관리하기 위해 자가 균형 이진 탐색 트리인 AVL 트리의 핵심 구조를 반영했다. 데이터가 한쪽으로 치우쳐 검색 성능이 오버헤드로 이어지는 것을 막기 위해 균형 인수를 실시간으로 체크하는 로직을 구축했다. 나아가 LL, RR, LR, RL의 네 가지 회전 알고리즘을 소스코드로 직접 제어하고 복합 연동해 보면서, 자가 균형 트리가 트리 구조를 정형화하고 동적으로 정렬을 유지하는 메커니즘을 명확히 이해하게 되었다.

② 주에서

이번 프로젝트를 진행하면서 자료구조 수업에서 이론으로만 접했던 개념들을 직접 C 언어로 구현해보는 값진 경험을 할 수 있었다. 그 중에서도 이진 탐색 트리를 직접 구현하는 과정이 가장 인상 깊었다. 단순히 삽입과 탐색 연산을 넘어, 삭제 연산에서 후계자 노드를 찾아 트리 구조를 유지하는 과정을 직접 코드로 작성하면서 이진 탐색 트리가 단순한 개념이 아니라 정교한 설계가 필요한 자료구조임을 몸소 느낄 수 있었다. 특히 재귀 함수를 활용하여 트리를 순회하고 조작하는 과정에서 재귀적 사고 방식에 대한 이해가 크게 향상되었으며, 중위 순회를 통해 학생 데이터가 정렬된 순서로 출력되는 것을 직접 확인했을 때 자료구조의 실용적인 가치를 실감할 수 있었다.

해시 맵 구현 역시 이번 프로젝트에서 새롭게 학습한 핵심 내용이었다. 해시 함수를 직접 설계하고 체이닝 방식으로 충돌을 처리하는 코드를 작성하면서, $O(1)$ 이라는 탐색 속도가 단순히

주어지는 것이 아니라 해시 함수의 설계와 충돌 처리 방식에 따라 성능이 크게 달라질 수 있다는 점을 직접 체감할 수 있었다. 처음에는 해시 충돌이 발생했을 때 데이터가 유실되는 오류를 경험하기도 했는데, 이를 디버깅하고 Chaining 방식으로 해결하는 과정에서 오류를 스스로 분석하고 해결하는 능력이 크게 향상되었다. 좋은 해시 함수를 설계하는 것이 전체 시스템의 성능에 얼마나 큰 영향을 미치는지 이해하게 된 것이 가장 큰 수확이었다.

이진 탐색 트리와 해시 맵을 하나의 시스템 안에서 연동하여 동작하도록 설계하는 과정에서 자료구조를 단독으로 사용하는 것을 넘어 복합적으로 활용하는 능력을 키울 수 있었다. 계층형 트리로 학사 조직 구조를 표현하는 동시에 해시 맵으로 빠른 탐색을 지원하는 구조를 설계하면서, 하나의 문제를 해결하기 위해 여러 자료구조의 장점을 조합하는 방식으로 사고하는 법을 배울 수 있었다. 프로젝트의 대부분을 직접 구현하면서 때로는 어렵고 벅차기도 했지만, 그만큼 자료구조 전반에 대한 이해도가 크게 향상되었고 문제를 스스로 분석하고 해결해 나가는 능력이 성장했음을 느낄 수 있었던 의미 있는 프로젝트였다.